

Project Paper

RoboBirder: Task T2s

Development of Intelligent Systems - Task T2s

Team Pi:

Maj Bregar

Rok Hladin

Matjaž Pogačnik

June 2025

Mentor: prof. dr. Danijel Skočaj

Faculty of Computer and Information Science

University of Ljubljana

Contents

1	Introduction	4
2	Methods	5
2.1	Adaptive Monte Carlo Localization (AMCL)	5
2.2	Simultaneous Localization and Mapping (SLAM)	5
2.3	Harris Corner Detection	5
2.4	Canny Edge Detection	5
2.5	Visual Transformer (ViT)	5
2.6	Coordinate Transforms in ROS 2	6
2.7	Random Sample Consensus (RANSAC)	6
2.8	Hough Transform	6
2.9	Otsu's Thresholding	6
2.10	YOLO	6
2.11	ResNet18	6
2.12	Speech-to-Text and Text-to-Speech	6
2.12.1	Speech-to-Text	7
2.12.2	Text-to-Speech	7
3	Implementation and Integration	7
3.1	Autonomous Navigation	7
3.2	Finetuning ResNet-18 for Bird Species Classification	8
3.3	Bird Detection and Classification	8
3.4	Ring Detection	10
3.5	Face Detection and Gender Recognition	12
3.6	Human-Robot Interaction (Speech)	13
3.7	Bridge Crossing	14
3.8	ROS Node Architecture	16
3.9	Communication and Data Flow	17
3.10	Software Tools and Libraries	19
4	Real Robot Implementation (Task 1R)	19
4.1	Implementation on the Real Robot	19
4.1.1	Autonomous Exploration	20
4.1.2	Face Detection	20
4.1.3	Navigation and Interaction	20
4.1.4	Attempted Ring Detection	21
4.2	Results with the Real Robot	21
5	Division of Work	21
6	Results	22
6.1	Evaluation Run	22
6.2	Module Performance	22
7	Conclusion	23
A	List of Target Bird Species	24
B	Generated bird catalogue	25

List of Figures

1	RViz map with navigation waypoints (blue vectors) calculated using the skeletonization-based exploration strategy.	8
2	Bird detection helper window showing YOLO detections.	9
3	Example of a detected and classified American Crow using ResNet18.	9
4	Example of the generated Bird catalogue.	10
5	Ring detection helper window showing detected ellipses.	12
6	Face detection and gender recognition examples.	13
7	The dialogue interface for human-robot interaction.	14
8	Bridge helper view showing line detection for navigation.	16
9	RViz view showing the robot's map and useful markers.	23

1 Introduction

The robot must autonomously explore a simulated park environment while performing a series of perception, navigation, and interaction tasks to complete the mission of taking photographs of birds and relaying the information about their location to the detected people. This task includes:

- **Autonomous Exploration and Mapping:**
 - Explore the park using SLAM to construct a map.
 - Autonomously navigate around the park using the constructed map.
 - Avoid restricted regions such as water and grass.
 - Divide the park into three sectors: East, Center, and West for systematic coverage.
- **Bridge Navigation:**
 - Detect and traverse narrow bridge structures using visual cues from the arm-mounted camera.
 - Search for a final red X marker on the island indicating a stopping point.
- **Ring Detection and Navigation:**
 - Detect colored 3D rings in the environment.
 - Use rings as waypoints to localize the areas where birds are likely to be found.
 - Classify ring colors (red, green, blue, black).
- **Bird Detection and Classification:**
 - Detect all 24 flat bird images placed on the rods holding up the rings.
 - Recognize the species of each bird.
 - Move close to each detected bird, orient the camera, and take a photograph.
 - Generate a final PDF report listing all detected and classified birds.
- **Human Detection and Interaction:**
 - Detect and approach humans in the park.
 - Identify each person's gender.
 - Engage in a dialogue to determine their favorite bird species:
 - * Women state their favorite bird directly.
 - * Men require the robot to ask multiple times; the bird repeated twice or confirmed with "yes" is accepted as the favorite.
 - Use previously gathered map and bird data to make conversation context-aware.
- **Task Completion:**

- Complete the bird and human interaction objectives.
- Reach the final goal marked with a red X.

Our team, "Pi," focused on developing modular solutions for each sub-problem and integrating them into a cohesive system using the Robot Operating System (ROS). This report outlines the methods employed, the implementation details, the performance of our system, the division of work among team members, and concludes with a discussion of the challenges faced and lessons learned.

2 Methods

This section describes the core methods used in our system.

2.1 Adaptive Monte Carlo Localization (AMCL)

AMCL is a probabilistic localization algorithm that uses a particle filter to track the pose of a robot against a known map [8]. It updates particle weights based on laser scan matching and odometry motion models. In our simulation, AMCL was used for global localization within a pre-built occupancy grid map during inspection tasks.

2.2 Simultaneous Localization and Mapping (SLAM)

SLAM algorithms enable a robot to build a map of an unknown environment while simultaneously determining its location within that map [5]. In our simulation, SLAM (via `slam_toolbox`) was used to generate the initial map of the environment before running AMCL-based inspection runs.

2.3 Harris Corner Detection

Harris corner detection is a classical method for identifying interest points (corners) in an image by analyzing changes in intensity gradients [9]. In our simulation, it was used in both `detect_rings.py` and `bridge_navigation.py`. In both cases, it served to remove corner features from edge maps, which helped to isolate cleaner contour lines for rings and bridge guardrails, respectively.

2.4 Canny Edge Detection

Canny edge detection is a multi-stage algorithm that detects edges by applying Gaussian smoothing, gradient calculation, non-maximum suppression, and hysteresis thresholding [2]. In our simulation, it was used as part of the pre-processing pipeline in `detect_rings.py` to detect ring edges and in `bridge_navigation.py` to identify the bridge guardrails.

2.5 Visual Transformer (ViT)

Visual Transformers [3] adapt Transformer architectures to image processing by representing images as sequences of patches. In our simulation, a ViT-based model was used in the face detection pipeline (`face_detector.py`) to classify the gender of detected human faces.

2.6 Coordinate Transforms in ROS 2

Coordinate transforms are essential for expressing sensor data and robot poses in consistent frames of reference [7]. In our simulation, the `tf2` library was used extensively in `detect_people.py`, `bird_detector.py`, `bridge_navigation.py` and `detect_rings.py` to transform 3D positions from camera frames to the global `map` frame for navigation and inspection.

2.7 Random Sample Consensus (RANSAC)

RANSAC is a robust model fitting algorithm that iteratively selects random data subsets to estimate model parameters and identify inliers [6]. In our simulation, RANSAC was employed to fit planes to point clouds around detected faces and rings, allowing estimation of surface normals for viewpoint planning.

2.8 Hough Transform

The Hough transform is a feature extraction technique used to detect parameterized shapes (e.g., lines, circles) in images [4].

2.9 Otsu’s Thresholding

Otsu’s method automatically determines an optimal global threshold to separate foreground and background by maximizing inter-class variance [12]. In our simulation, Otsu’s thresholding was applied to pre-processed grayscale images in `detect_rings.py` and `bridge_navigation.py` to enhance ring visibility before edge and feature extraction.

2.10 YOLO

YOLO is a real-time object detection algorithm that treats object detection as a single regression problem, directly predicting bounding boxes and class probabilities from entire images in one evaluation [13]. Its unified architecture enables fast and accurate detection. In our simulation, a general YOLOv8n model, obtained from Ultralytics, was used to detect faces, and birds from RGB camera images.

2.11 ResNet18

ResNet-18 (Residual Network) was used in classification system for birds. It is a deep convolutional neural network that leverages residual connections to enable effective training of relatively deep models, mitigating the vanishing gradient problem and facilitating the learning of complex visual features [10].

2.12 Speech-to-Text and Text-to-Speech

The human-robot dialogue system, implemented in `dialogue.py`, is built upon two core speech technologies: Speech-to-Text (STT) for understanding user input and Text-to-Speech (TTS) for generating the robot’s verbal responses.

2.12.1 Speech-to-Text

For STT, our system uses the `SpeechRecognition` library [14], which serves as a wrapper for various online and offline speech recognition engines. We configured it to use the Google Speech Recognition API, which leverages large-scale deep learning models to achieve high transcription accuracy.

2.12.2 Text-to-Speech

For TTS, we integrated the `pyttsx3` library [1], a platform-independent, offline TTS engine. This choice ensured that the robot could vocalize its responses without requiring an active internet connection, making the interaction more robust. The library synthesizes speech directly on the machine, allowing for real-time generation of the robot’s questions and statements during the dialogue.

3 Implementation and Integration

This section details how the aforementioned methods were implemented and integrated into a distributed robotic system using ROS.

3.1 Autonomous Navigation

Our navigation system integrates SLAM-based mapping, dynamic path planning, and robust localization using standard ROS 2 tools.

- **Mapping:** Utilizing SLAM (Simultaneous Localization and Mapping) techniques, such as GMapping, to build a 2D occupancy grid map of the simulated environment as we drive the robot.
- **Path Planning:** Our robot performs path planning using the ROS 2 Navigation Stack (Nav2), which is responsible for computing safe and efficient paths to target locations. Path planning is initiated when `RobotCommander` node sends a goal to the `NavigateToPose` action server, provided by the `nav2_bt_navigator` node. Internally, `nav2_bt_navigator` runs a Behavior Tree that coordinates the overall navigation process. It uses the `nav2_planner` node, which performs global path planning using graph-based search algorithms such as A* or Dijkstra’s algorithm on a 2D occupancy grid. Once a global path is found, the `nav2_controller` node handles local path following and dynamic obstacle avoidance using a local planner. In our architecture, `RobotCommander` acts as a high-level client, while the actual path planning, map updates, and path execution are handled by Nav2 nodes and core ROS 2 action mechanisms.
- **Exploration Strategy:** To ensure comprehensive coverage of the environment, we compute navigation waypoints from the acquired map. This is done by fitting lines using Hough transform and slicing the fitted lines into segments of 0.2 meters. For each segment a navigation waypoint is set as a point 0.7 meters from the middle of the line segment to each side of the line. If the location is illegal based on the map, it is removed. If the generated waypoint is too close to any adjacent wall for safe navigation of the robot, it is moved away perpendicular to that wall, ensuring 0.5 meter

clearance of each waypoint. Each navigation waypoint also has a direction which points towards the wall. The robot navigator then visits these goals in greedy way, selecting the closest goal based on A* distance.

- **Localization:** Continuous localization using AMCL (Adaptive Monte Carlo Localization) was used in all parts of the task to track the robot’s pose within the map accurately.

The ‘planner.py’ and ‘navigator.py’ scripts encapsulate these functionalities.

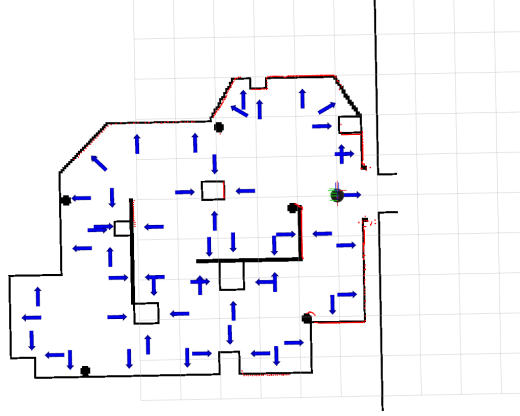


Figure 1: RViz map with navigation waypoints (blue vectors) calculated using the skeletonization-based exploration strategy.

3.2 Finetuning ResNet-18 for Bird Species Classification

We fine-tune ResNet-18 on CUB_200_2011 dataset originally containing 200 classes of birds. For our task we used a subset of 24 classes A. Input images were resized to 224x224 pixels, augmented using random horizontal flips and color jitter, and normalized before being passed to the network. The model is trained using the Adam optimizer with a learning rate of 10^{-4} , and the cross-entropy loss:

$$\mathcal{L}_{\text{class}} = - \sum_{i=1}^N y_i \log(\hat{y}_i)$$

3.3 Bird Detection and Classification

The task required detecting and classifying multiple bird species.

- **Initial Bird Detection (Candidate Finding):** We use a general YOLOv8n model to perform fast detection of "bird" candidates in the robot’s camera feed. Input RGB images are pre-filtered using depth information to focus on closer objects. This stage helps in identifying potential locations of birds for closer inspection.
- **Species Classification and Confirmation:** Once YOLOv8n detects an object classified as "bird" with classification accuracy above 0.7 (triggered by a state change, e.g., "ARRIVED_AT_BIRD"), while also standing at a

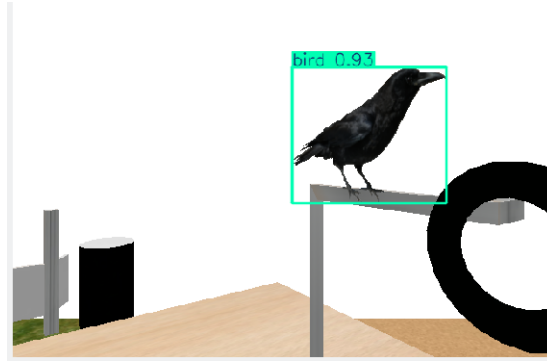


Figure 2: Bird detection helper window showing YOLO detections.

navigation waypoint, it switches to a state for closer inspection. The regions of interest identified by YOLO are cropped from the RGB image and passed to a fine-tuned ResNet18 convolutional neural network for classification into one of 24 distinct species. A successful classification is published when enough classifications with score above 0.4 are made for the associated bird. The classification performance differs between using **ogre** and **ogre2** for simulation engine, as does therefore the required number of detections. In our experience, the performance was better on **ogre2**, as it supports the sky, providing better illumination. The final class is determined as the class with the highest mean classification accuracy for the associated bird. The classifications are assigned to a bird (a group of detections) based on proximity (< 0.5 meters) to the center of a known group of detections (initially published on `/bird_initial_position_detection` topic). The 3D position of the bird is determined using the point cloud data associated with the middle of the cropped bird image. Confirmed birds are published as `DetectedBird` messages to the `/detected_birds` topic, including species name, confidence, and 3D map coordinates.



Figure 3: Example of a detected and classified American Crow using ResNet18.

- **Bird catalogue:** On successful classification a PDF Bird catalogue is generated (or updated) with a picture and species of the detected bird. For that **ReportLab** Python library was used.

Report Golob



Figure 4: Example of the generated Bird catalogue.

3.4 Ring Detection

Ring detection is performed using a series of synchronized image processing and 3D analysis steps implemented in the `ring_detector.py` node. The pipeline is triggered only when the robot reaches specific states such as `ARRIVED_AT_GOAL` or `MOVING_CLOSER_TO_BIRD`. The key steps are:

- **Input Synchronization:** The detector subscribes to synchronized RGB images, depth images, and point clouds from the top arm-mounted camera.
- **Preprocessing:** The RGB image is converted to HSV color space, and the depth image is normalized and clamped to 2.0 meters to focus detection on nearby rings. This constraint improves reliability by ignoring distant, noisy shapes.
- **Edge and Corner Filtering:**
 - **Edge Detection:** The depth image is passed through a Canny edge detector with thresholds of 50 and 150.
 - **Corner Detection:** Harris corner detection (block size 5, kernel size 3, $k=0.1$) is applied to the Canny edge map to identify and remove edge regions that belong to corners. This helps separate overlapping objects, such as birds close to rings and the rods holding the rings in place.

- **Edge Spur Removal:** Small spurs (up to 25 pixels long) are pruned by tracing from edge endpoints, reducing leftover artifacts by imprecise harris corner removal.
- **Contour and Ellipse Detection:**
 - Contours are extracted from the cleaned edge image.
 - Contours shorter than 10 points or too linear (with $>85\%$ of points lying close to a fitted line) are discarded.
 - Remaining contours are fitted with ellipses (only if they have ≥ 9 points).
- **Ring Candidate Filtering:**
 - To identify actual ring shapes, the algorithm considers all pairs of fitted ellipses and tests whether one could be the inner and the other the outer boundary of the same ring.
 - Each pair is evaluated using several geometric constraints to filter out false positives:
 - * The distance between the centers of the two ellipses must be ≤ 1.0 pixel to ensure they are concentric.
 - * The difference in orientation angles between the ellipses must be $\leq 40^\circ$ to enforce shape alignment.
 - * The difference in their minor axes (used to approximate ring thickness) must fall between 2 and 50 pixels.
 - * Both ellipses must have a minimum minor axis of at least 4 pixels to ensure they are not degenerate or too flat.
 - * The aspect ratio (major/minor axis) of each ellipse must be ≤ 1.5 , enforcing that the ellipses are roughly circular.
 - * The difference between their border thicknesses (computed from major and minor axes) must be ≤ 5 pixels, ensuring visual symmetry of the ring.
 - After satisfying these 2D constraints, each candidate undergoes a 3D depth validation step. Specifically, 10 radial depth samples are taken inside the inner ellipse (center region) and between the two ellipses (ring region). The ring is accepted only if the median depth of the center is at least 1.5 meters greater than the ring border. This confirms that the ring is a hollow structure suspended in space (i.e., a real 3D ring), not a ring drawn or printed on a flat surface.
- **Color Classification:**
 - A binary ring mask is created from the area between the inner and outer ellipses, slightly eroded to reduce noise, and applied to the HSV image.
 - A 180-bin hue histogram is computed over the masked region, smoothed with a circular Gaussian ($\sigma = 3$), and normalized.
 - The histogram is compared to predefined reference histograms for red (0°), green (60°), and blue (120°) using cosine similarity.

- If the average V (value) channel in the ring region is below 0.35, the ring is classified as **black**; otherwise, the color with the highest similarity score is selected.

- **Ring Grouping and Publishing:**

- Ring detections are grouped if they occur within 0.5 meters of each other.
- Groups with at least 3 observations are used to determine the average position and majority color.
- Published ring detections (`DetectedRing` messages on `/ring_position`) include timestamp, color, and 3D map coordinates.
- Previously published detections are tracked and not duplicated.

The `ring_detector.py` script is responsible for these operations.

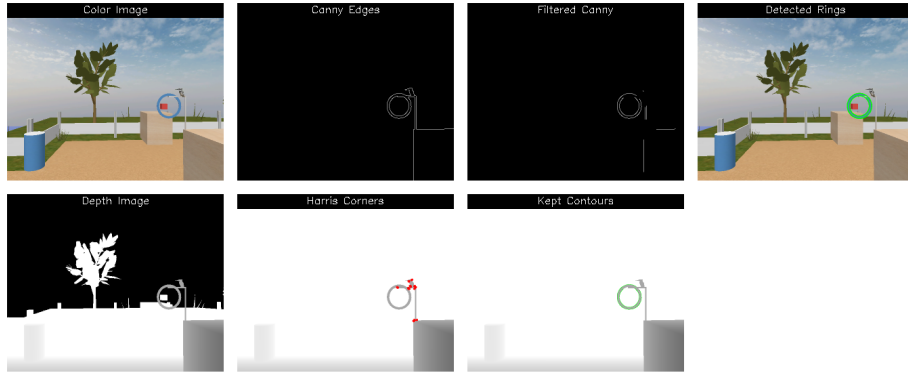


Figure 5: Ring detection helper window showing detected ellipses.

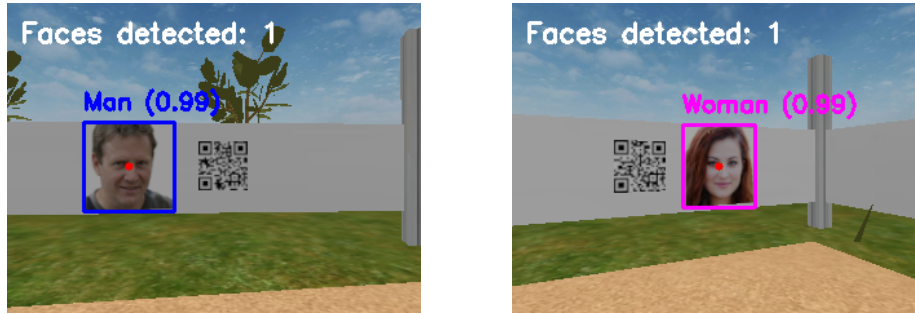
3.5 Face Detection and Gender Recognition

For human-robot interaction, our system first detects human faces and then attempts to recognize their gender. This process utilizes synchronized RGB images and point cloud data.

- **Face Detection:** We use a general YOLOv8n (You Only Look Once) model [13], obtained from Ultralytics, to detect human faces (class 0) in the RGB camera feed (from `/oakd/rgb/preview/image_raw`). Detections are filtered based on a confidence threshold (typically 0.6).
- **Gender Recognition:** Once a face is detected and its bounding box extracted, the cropped face region is passed to a pre-trained Vision Transformer (ViT) model for gender classification. Specifically, we use the `dima806/fairface_gender_image_detection` model available on Hugging Face [11]. This model classifies the face as "man" or "woman" with reported accuracy around 93.4%. The implementation uses the `transformers` library by Hugging Face.

- **3D Localization and Normal Estimation:** For each valid face detection (passing depth checks, e.g., within 1.5m), its 3D position in the camera frame is determined using the synchronized point cloud data (from `/oakd/rgb/preview/depth/points`). A small window of points around the face's 2D center is extracted from the point cloud. RANSAC-based plane fitting is applied to these points to estimate the 3D centroid of the facial surface and its normal vector. This normal is oriented to face the camera, and an interaction point is calculated at a short offset along this normal.

The `face_detector.py` script manages this pipeline, including grouping multiple detections of the same individual, transforming coordinates to the map frame, and publishing confirmed detections.



(a) Detection of a male face.

(b) Detection of a female face.

Figure 6: Face detection and gender recognition examples.

3.6 Human-Robot Interaction (Speech)

Effective communication with simulated humans is achieved through a dedicated dialogue module (`dialogue.py`) that integrates a graphical user interface (GUI), speech-to-text (STT), text-to-speech (TTS), and natural language understanding (NLU) capabilities.

- **Graphical User Interface (GUI):** A `tkinter`-based chatbox (`ChatboxGUI`) provides a visual interface for the dialogue, displaying conversation history and allowing users to type responses or trigger voice input. It also handles microphone device selection, prioritizing 'pulse' audio devices for better compatibility.
- **Speech-to-Text (STT):** The `speech_recognition` library, configured to use the Google Speech Recognition API, converts spoken user input (captured via microphone) into text.
- **Text-to-Speech (TTS):** The `pyttsx3` library is used to synthesize robot speech, providing audible responses to the user.
- **Natural Language Understanding (NLU):**
 - Recognized user text is processed by the `SpeechResult` class.

- **spaCy** (using the `en_core_web_sm` model, if available) is employed for basic NLP tasks.
- Bird species are extracted from user utterances by matching against a predefined list (`BIRD_SPECIES`) and associated keywords (`BIRD_KEYWORDS`).
- User responses are also checked for positive (e.g., "yes", "sure") and negative (e.g., "no", "nope") keywords to understand affirmations or negations.

- **Dialogue Flow (`BirdDialogue` class):**

- The dialogue is gender-specific, initiated based on the gender information received (e.g., from the `face_detector_node`).
- **Dialogue with Women:** The robot asks for their favorite bird. If a bird is successfully extracted, its location is described.
- **Dialogue with Men:** The robot asks for their favorite bird and then enters a confirmation loop ("Are you sure?"). The user can confirm, or state a different bird. The robot proceeds once a bird choice is confirmed or a new bird is mentioned multiple times.
- **Location Description:** If a bird is identified, its location is described in relation to the park's regions ("West", "Center", "East" based on coordinates) and the color of the nearest ring, using pre-supplied lists of known bird and ring locations.

This system is designed to be invoked when an interaction is required, taking the detected gender and current environmental knowledge (bird/ring locations) as input.

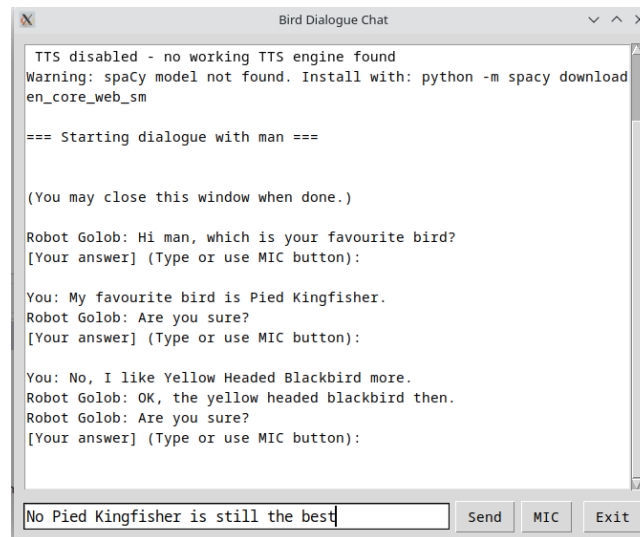


Figure 7: The dialogue interface for human-robot interaction.

3.7 Bridge Crossing

Navigating the bridge involves robust visual perception and dynamic pose estimation using the robot's top-mounted arm camera. The processing pipeline

adapts based on the robot's situation and environment, and proceeds through the following stages:

- **Image Preprocessing:** The RGB input from `/top_camera/rgb/preview/image_raw` is converted to HSV. The Hue channel is blurred and thresholded using Otsu's method, then inverted to highlight the bridge structure. This binary image serves as the foundation for further processing.
- **Guardrail Detection and Path Guidance:**
 - **Edge and Corner Removal:** Canny edge detection is applied. Harris corners are detected and removed from the upper third of the image and, if needed, the entire image to suppress noise from bridge-land transitions.
 - **Component Analysis and Line Fitting:** The two largest edge components are selected. Each is segmented into vertical slices, and line segments are fit using `cv2.fitLine`. Metadata including endpoints, angles, parent edge ids and centers is stored.
 - **Line Pairing and Filtering:** Pairs of left and right lines are formed based on angular similarity ($\leq 60^\circ$), spatial separation (20–80% of image width), and different component IDs. Pairs with abrupt direction changes ($> 35^\circ$) are filtered out.
 - **Pose Calculation:** For the final valid pair, the midpoint is computed and a forward direction vector is inferred. This 2D image pose is transformed into a 3D pose in the `map` frame using synchronized point cloud data from `/top_camera/rgb/preview/depth/points`.
- **End-of-Bridge Detection:** When crossing from the water bridge onto land, visual segmentation temporarily fails. To handle this, the system checks if the top 20 rows of the binary image are fully white. If so, it assumes the robot is at the edge of the bridge and blindly advances. A forward pose is computed from a fixed point in the image and transformed to the `map` frame for navigation.
- **Red X Marker Detection:** After reaching land, the robot seeks a red X stop marker indicating the final goal location.
 - The lower central portion of the image is masked to avoid the robot's body.
 - The remaining binary mask is eroded and thresholded with Otsu's algorithm again to isolate colored regions on the path.
 - Connected components are analyzed, and for large enough blobs, hue histograms are computed and compared to a red reference histogram using cosine similarity.
 - If similarity exceeds 0.75, the blob is accepted as a red X, and its center point is projected into a final 3D pose for navigation.

The `bridge_navigation.py` node publishes the resulting pose as a `BridgePose` message to the `/bridge_pose_map` topic, tagging it as a final goal if a red X is

detected. The node is active only when the robot is in the `BRIDGE_NAVIGATION` or `GO_TO_FINAL_POSITION` state, unless manually overridden.

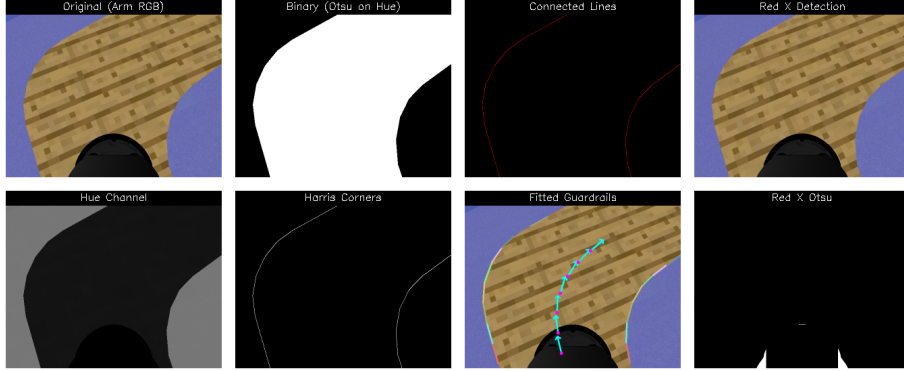


Figure 8: Bridge helper view showing line detection for navigation.

3.8 ROS Node Architecture

Our system is architected as a set of loosely coupled ROS nodes, each responsible for a specific functionality. This modular design facilitates development, testing, and debugging. Key nodes include:

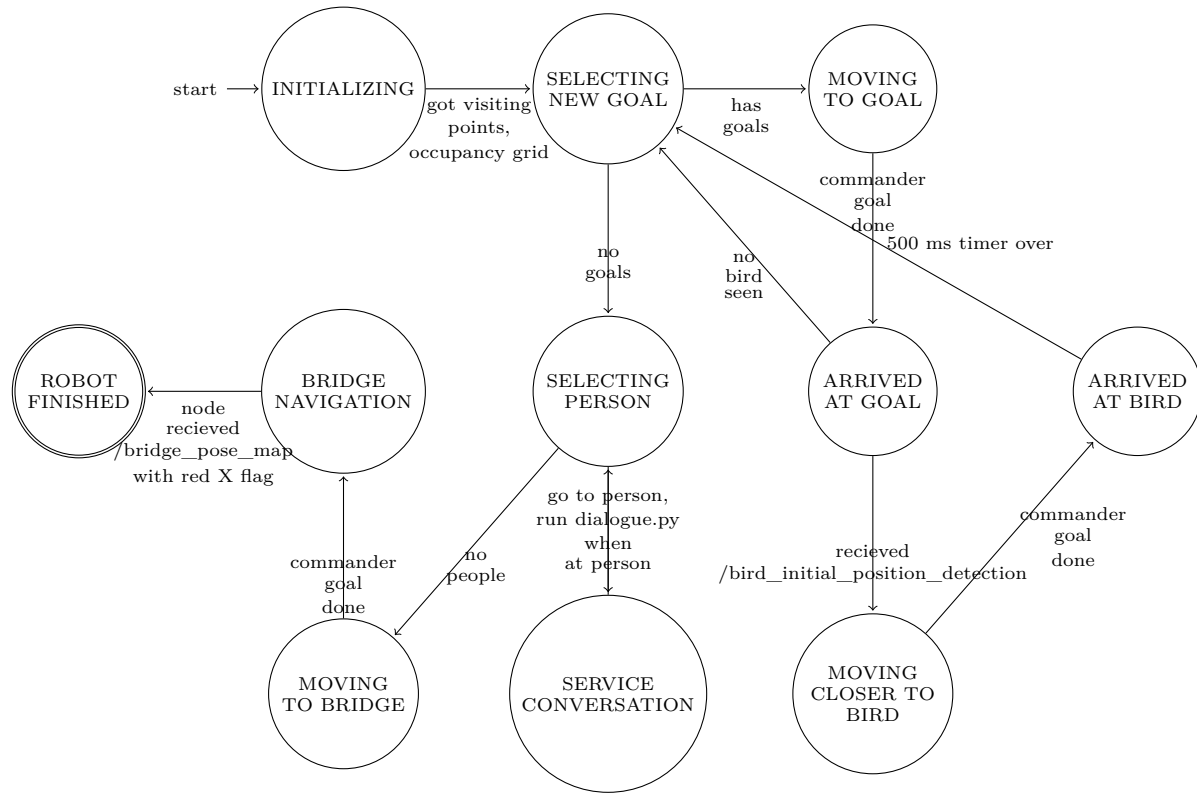
- **navigator** (from `navigator.py`): This is the central navigation node that controls the robot state and its actions.
- **bird_detector** (from `bird_detector.py`): This node implements the two-stage bird detection and classification pipeline.
- **ring_detector** (from `ring_detector.py`): Processes camera images to detect rings and their colors. Publishes ring information.
- **face_detector** (from `face_detector.py`): This node handles the detection of human faces and subsequent gender classification.
- **bridge_navigation** (from `bridge_navigation.py`): This node is responsible for visually navigating the bridge.
- **planner** (from `planner.py`): This node publishes the robot visiting goals once during the initialization phase.
- **arm_mover** (from `arm_mover_actions.py`): This node moves the top arm-mounted camera upon receiving a position on `/arm_command`.

3.9 Communication and Data Flow

The core logic of the robot is implemented in the main navigation node (`navigator.py`), which is responsible for coordinating all high-level behavior throughout the task. The navigation node subscribes to several topics to receive information from perception and planning nodes. This data is used to update internal structures such as the list of known birds, rings, and people to interact with.

- `/detected_faces` (from `face_detector`): New face groups are added to an internal queue for future person interactions.
- `/ring_position` (from `ring_detector`): Detected ring groups are stored in a list used for bird localization during person interactions.
- `/detected_birds` (from `bird_detector`): Detected and classified birds are added to an internal list for future person interactions.
- `/bird_initial_position_detection` (from `bird_detector`): Sent by the bird detection node when a bird is first spotted at a goal.
- `/inspection_markers` (from `planner`): During initialization, a list of exploration waypoints is received and stored in a goal queue.
- `/map` (from `map server`): Provides the static occupancy grid map for navigation and costmap generation.
- `/global_costmap/costmap` (from `nav2`): Supplies dynamic obstacle information used in goal planning and collision avoidance.
- `/initialpose` (published): Sends an initial pose estimate to AMCL during robot initialization.
- `/robot_internal_state` (published): Broadcasts the current internal FSM state of the robot for node behaviour synchronization.
- `/arm_command` (published): Sends string commands to control the robot's arm behavior.
- `/amcl_pose` (from `AMCL`): Provides the robot's estimated position for localization.
- `/bridge_pose_map` (from `bridge_navigation`): Used to identify the final waypoint on the bridge, marked with a red X.

The navigation node maintains the robot's current operational state via a finite state machine, guiding the robot through stages such as exploring the park, approaching birds, interacting with humans, and crossing the bridge. That state is being broadcast with the topic `/robot_internal_state`.



- **INITIALIZING:**
The robot publishes the initial pose to AMCL and waits to receive the occupancy grid and visiting points from the map and planner, publishes an arm command to `/arm_command` reposition the arm-mounted camera for bird and ring detection.
- **SELECTING NEW GOAL:**
Selects a new visiting goal if any are still available, using A* to choose the nearest one.
- **MOVING TO GOAL:**
A passive state in which the robot is navigating toward the selected inspection goal pose.
- **ARRIVED AT GOAL:**
Waits 500 ms to check whether any `/bird_initial_position_detection` messages have been received.
- **MOVING CLOSER TO BIRD:**
Upon receiving a bird position, the robot searches the global costmap for a reachable point at a fixed distance from the bird and moves to it.
- **ARRIVED AT BIRD:**
Waits 500 ms to see if any additional bird classifications are published.
- **SELECTING PERSON:**
Selects the most recently detected person from the internal person queue.

- **SERVICE CONVERSATION:**
Navigates to the selected person and launches `dialogue.py` to begin interaction.
- **MOVING TO BRIDGE:**
Travels to a hardcoded bridge start pose and, before arrival, publishes an arm command to `/arm_command` reposition the arm-mounted camera for bridge navigation.
- **BRIDGE NAVIGATION:**
Follows the sequence of poses published to `/bridge_pose_map`.
- **ROBOT FINISHED:**
Enters an idle state indicating all tasks have been completed.

3.10 Software Tools and Libraries

- **ROS2 Humble:** The primary framework for integration.
- **Python:** The main programming language for implementing the nodes.
- **OpenCV:** Extensively used for image processing tasks.
- **Deep Learning Libraries:**
 - **Ultralytics YOLO:** For the YOLOv8n object detection model.
 - **PyTorch** and **Torchvision:** For the ResNet18 image classification model.
- **SpeechRecognition:** For speech processing.
- **Standard ROS packages:** `gmapping_slam`, `amcl`, `move_base` (or custom navigation stack components), `tf2_ros`, `cv_bridge`, `sensor_msgs_py`, `message_filters`.
- **Hugging Face Transformers:** For loading and running the gender classification model.
- **ReportLab:** For generating PDF reports of detected birds.

4 Real Robot Implementation (Task 1R)

This section outlines the specific features and results from our work on the real TurtleBot 4 (Task 1R), which focused primarily on human detection and interaction.

4.1 Implementation on the Real Robot

While sharing a common ROS 2 framework with the simulation task, the real-robot implementation required several specific adaptations to handle real-world sensor data and constraints.

4.1.1 Autonomous Exploration

To achieve autonomous exploration, we implemented a coverage-based path planning strategy in the `planner.py` node. Instead of dynamic frontier-based exploration, this node generates a comprehensive set of inspection waypoints from a pre-existing map. The process involves:

1. Analyzing the 2D occupancy grid to identify wall structures. This is done by applying a Canny edge detector to the map, followed by a Hough Line Transform to extract straight wall segments.
2. Generating camera poses (waypoints) at a fixed offset from these detected walls. Waypoints are created on both sides of each wall segment to ensure full coverage.
3. Ensuring all generated waypoints are in navigable, free space and maintain a safe clearance from obstacles, which is verified using a distance transform computed on the map.
4. Publishing these waypoints as a `MarkerArray`, which the `face_search.py` navigation node then consumes to visit each point, thereby systematically exploring the entire area.

4.1.2 Face Detection

Instead of the resource-intensive YOLO model used in simulation, we employed a classical computer vision approach for face detection on the real robot. The `detect_people_haar.py` script utilizes OpenCV's Haar Cascade classifier, which provides a lightweight and fast method for detecting frontal faces. This choice was better suited to the computational resources of the robot's onboard computer.

The node processed RGB and depth streams from the OAK-D camera. For each detected face, it performed RANSAC-based plane fitting on the corresponding point cloud data to estimate the face's 3D position and surface normal. A key feature implemented for robustness was a planarity check, which helped distinguish between real human faces and pictures of faces on flat surfaces like walls.

4.1.3 Navigation and Interaction

The core navigation logic for the real robot was managed by the `face_search.py` node. This node would receive face detections, add them to a queue, and interrupt its current exploration task to navigate towards the person.

A crucial feature for real-world operation was the ability to compute a safe navigation goal. This was achieved by checking the local occupancy map around a detected face's coordinates and, if too close to an obstacle, calculating a "pushed" navigation goal at a safer distance from the wall. This ensured the robot could approach people without risking a collision. Once at the interaction goal, it used the same `pyttsx3`-based TTS engine as in the simulation to greet the person.

4.1.4 Attempted Ring Detection

For bonus points, we began implementing a ring detection module. The approach was similar to its simulation counterpart, relying on HSV color thresholding to segment potential ring candidates and a Canny edge detector to detect edges of rings. We could reliably detect most of the rings, but the problem was bad quality of depth image, therefore we could not determine the location of the rings.

4.2 Results with the Real Robot

Our system was successfully deployed on the real TurtleBot 4. The Haar Cascade detector was able to detect faces in real-time, although its performance was, as expected, more sensitive to lighting conditions and head orientation compared to the deep learning models used in simulation.

The navigation system reliably guided the robot to an appropriate interaction distance from detected individuals, with the safe navigation goal calculation proving effective in cluttered environments. The TTS-based interaction was also successfully demonstrated. The primary challenges encountered were related to the inherent noise and variability of real-world sensor data, which required more careful filtering and parameter tuning than in the controlled simulation environment.

5 Division of Work

The project was a collaborative effort by Team Pi. While we often discussed and debugged problems together, primary responsibilities for development and implementation were divided as follows:

- **Rok Hladin (32%):**
 - Developed the ring detection.
 - Implemented the human-robot interaction system, including face detection (YOLOv8n), 3D localization with normal estimation, and gender classification (using the FairFace ViT model [11]) for initiating dialogue.
 - Implemented speech-to-text (STT) and text-to-speech (TTS) functionalities with basic text processing within the dialogue manager.
- **Matjaž Pogačnik (34%):**
 - Developed the bird detection and classification system (YOLOv8n for detection, ResNet18 for species classification), including training/fine-tuning the classification model for 24 bird species, point cloud processing for 3D localization, and detection grouping.
 - Implemented the autonomous navigation and exploration system (path planning, AMCL localization).
 - Implemented the functionality for saving/reporting detected birds via ROS topics and PDF generation.

- **Maj Bregar (34%):**
 - Developed the bridge crossing module.
 - Led the overall system integration in ROS, designing the node architecture, message flows, and coordinating node interactions (state machine from `navigator.py`).
 - Addressed general debugging and system stability issues.

6 Results

6.1 Evaluation Run

During the evaluation run, the system performed reliably across most tasks. It successfully detected all human faces and all colored rings, and accurately localized birds relative to their associated rings. The dialogue system correctly simulated conversations and responded according to the person’s gender and preferences. Bridge traversal was also completed without issues, and the robot successfully followed the designated path to the final parking position marked by the red X.

One notable failure occurred during bird classification: a small bird (kingfisher) was not correctly classified due to insufficient frame generation. This issue was primarily caused by the high computational load and synchronizing RGB images, depth images, and point clouds, which lowered the detection rate and reduced the number of classification attempts. Additionally, the entire evaluation run took significantly longer than earlier test runs, as it was executed on a less powerful computer than the one used during development, amplifying the performance bottlenecks introduced by resource-intensive modules.

6.2 Module Performance

- **navigator** (from `navigator.py`): Functioned reliably, using A* for path planning. However, it is not optimal for multi-goal planning (e.g., visiting all birds), where solving a Traveling Salesman Problem (TSP) would be better, though too computationally intensive for our system.
- **bird_detector** (from `bird_detector.py`): Generally accurate, but occasionally failed to classify smaller birds due to limited resolution. The module is resource-heavy due to simultaneous use of YOLO and ResNet, and the need for high-resolution camera input.
- **ring_detector** (from `ring_detector.py`): Performed reliably in detecting and validating ring geometry and color. However, the use of high-resolution images combined with geometric and depth-based validation made it computationally demanding.
- **face_detector** (from `face_detector.py`): Detected faces accurately and performed well overall. Gender classification relied on a transformer-based model, which was computationally expensive.

- `bridge_navigation` (from `bridge_navigation.py`): Functioned correctly under most conditions. However, noisy AMCL localization estimates sometimes resulted in misaligned navigation goals.
- `dialogue` (from `dialogue.py`): Limited interaction due to poor microphone quality. The system relied on simple keyword-based recognition, which occasionally failed to detect bird names correctly due to lack of context awareness.

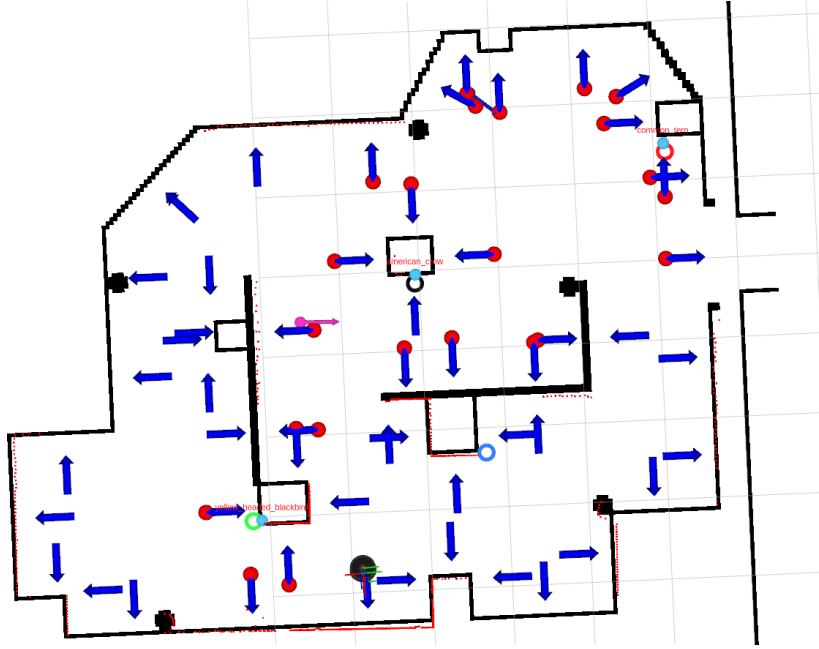


Figure 9: RViz view showing the robot's map and useful markers.

7 Conclusion

Team Pi successfully developed an integrated robotic system capable of addressing the multifaceted challenges of Task T2s within the bird park simulation. Our robot demonstrated capabilities in autonomous navigation, bird, ring and face detection and recognition, human-robot interaction via speech and gender detection, and specialized maneuvers like bridge crossing.

Throughout this project, we encountered several technical challenges:

- **Simulation Environment Quirks:** We faced issues with the Gazebo simulation environment. Initially, the Ogre rendering engine did not render the sky, requiring a switch to Ogre2. We also had to debug a problem where point cloud coordinates were unexpectedly flipped by 90 degrees in a configuration file, which complicated initial setup and localization efforts.
- **Resource Limitations:** Development was constrained by the need to use lab computers, as running the full simulation and ROS environment

on personal laptops was often not feasible. This limited our access and sometimes slowed down development and testing cycles. Additionally, the one computer we had available for local testing was significantly more powerful than the lab machines, which led to compatibility and performance issues when switching to less capable systems, as configurations were not optimized for them.

- **Integration Complexity:** Integrating multiple perception, navigation, and interaction modules into a reliably functioning system is inherently complex. Ensuring robust data flow, timing, and error handling across different ROS nodes required significant effort.
- **Synchronization Issues:** Several perception modules, especially those that estimate 3D positions using point cloud data, required strict synchronization of RGB images, depth images, and point clouds. Without synchronized messages, even minor delays could result in mismatched frames, leading to large spatial errors when estimating object positions. We used ROS 2 message filters to address this issue and ensure time-aligned data across sensors. However, this synchronization introduced a trade-off: since all detector inputs had to be synchronized, the overall detection framerate was reduced, as the system had to wait for all relevant sensor messages to arrive before processing.

Despite these challenges, the project was a valuable learning experience, providing deep insights into the practical aspects of developing intelligent robotic systems. We were able to apply theoretical knowledge to solve concrete problems and work collaboratively to build a complex system. Future work could focus on enhancing the robustness of individual modules, particularly improving the accuracy of classification tasks under varying conditions.

A List of Target Bird Species

The bird species classification module (ResNet18) was trained to identify the following 24 distinct species:

1. 002.Laysan_Albatross
2. 012.Yellow_headed_Blackbird
3. 014.Indigo_Bunting
4. 025.Pelagic_Cormorant
5. 029.American_Crow
6. 033.Yellow_billed_Cuckoo
7. 035.Purple_Finch
8. 042.Vermilion_Flycatcher
9. 048.European_Goldfinch
10. 050.Eared_Grebe

11. 059.California_Gull
12. 068.Ruby_throated_Hummingbird
13. 073.Blue_Jay
14. 081.Pied_Kingfisher
15. 095.Baltimore_Oriole
16. 101.White_Pelican
17. 106.Horned_Puffin
18. 108.White_necked_Raven
19. 112.Great_Grey_Shrike
20. 118.House_Sparrow
21. 134.Cape_Glossy_Starling
22. 138.Tree_Swallow
23. 144.Common_Tern
24. 191.Red_headed_Woodpecker

Note: The system is configured for these 24 species, as reflected in the classification model architecture ('resnet.fc' output features) and training data structure used in `bird_detector.py`.

B Generated bird catalogue

Report Golob



Common_Tern



American_Crow



Yellow_headed_Blackbird



Pied_Kingfisher

References

- [1] Natesh M. Bhat. *pyttsx3*. <https://pypi.org/project/pyttsx3/>. 2019.
- [2] John Canny. “A Computational Approach To Edge Detection”. in *Pattern Analysis and Machine Intelligence, IEEE Transactions on*: PAMI-8 (december 1986), **pages** 679–698. DOI: 10.1109/TPAMI.1986.4767851.
- [3] Alexey Dosovitskiy **and others**. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.
- [4] Richard O. Duda **and** Peter E. Hart. “Use of the Hough transformation to detect lines and curves in pictures”. in *Commun. ACM*: 15 (1972), **pages** 11–15. URL: <https://api.semanticscholar.org/CorpusID:1105637>.
- [5] Hugh Durrant-whyte **and** Tim Bailey. “Simultaneous localization and mapping: Part I”. in *Robotics and Automation Magazine, IEEE*: 13 (july 2006), **pages** 99–110. DOI: 10.1109/MRA.2006.1638022.
- [6] Martin A. Fischler **and** Robert C. Bolles. “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography”. in *Commun. ACM*: 24 (1981), **pages** 381–395. URL: <https://api.semanticscholar.org/CorpusID:972888>.
- [7] Tully Foote. “Tf: The transform library”. in *april* 2013: **pages** 1–6. ISBN: 978-1-4673-6223-8. DOI: 10.1109/TePRA.2013.6556373.
- [8] Dieter Fox **and others**. “Monte Carlo Localization: Efficient Position Estimation for Mobile Robots”. in *january* 1999: **pages** 343–349.
- [9] Christopher G. Harris **and** M. J. Stephens. “A Combined Corner and Edge Detector”. in *Alvey Vision Conference*: 1988. URL: <https://api.semanticscholar.org/CorpusID:1694378>.
- [10] Kaiming He **and others**. “Deep Residual Learning for Image Recognition”. in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*: 2016, **pages** 770–778. DOI: 10.1109/CVPR.2016.90.
- [11] D. Kim. *FairFace Gender Image Detection Model*. Hugging Face Model Repository. [Online]. Available: https://huggingface.co/dima806/fairface_gender_image_detection. Accessed: June 2025. 2023.
- [12] Nobuyuki Otsu. “A Threshold Selection Method from Gray-Level Histograms”. in *IEEE Trans. Syst. Man Cybern.*: 9 (1979), **pages** 62–66. URL: <https://api.semanticscholar.org/CorpusID:15326934>.
- [13] Joseph Redmon **and others**. “You only look once: Unified, real-time object detection”. in *Proceedings of the IEEE conference on computer vision and pattern recognition*: 2016, **pages** 779–788.
- [14] Anthony Zhang. *SpeechRecognition*. <https://pypi.org/project/SpeechRecognition/>. 2017.